

ADR-002 — RBAC à 6 rôles avec helper

`check_ati_access`

- **Statut** : Accepté
- **Date** : 2026-04-27
- **Auteur** : Jean Baptiste MBA NDONG
- **Décideurs concernés** : Concepteur, futur Directeur des Systèmes d'Information du Ministère
- **Référence code** : `backend/app/core/auth.py` (énum `Role`),
`backend/app/routers/ati.py` (fonctions `check_ati_access` , `_user_role_values`),
`docs/rbac_matrix.md`

Contexte

La PNPI met en œuvre un workflow administratif complexe d'octroi des Agréments Techniques Industriels (ATI) qui implique des **rôles institutionnels** hétérogènes :

- le **ministre** (signature finale, décisions d'appel),
- le **directeur** (validation hiérarchique, arbitrage),
- les **instructeurs** (instruction du dossier, demande de pièces),
- les **inspecteurs** (visites de conformité sur site),
- les **opérateurs industriels** (déposants),
- les **administrateurs techniques** (gestion des comptes, audit, exports).

Deux modèles de contrôle d'accès sont possibles :

1. **ACL fines** (Access Control Lists) — chaque ressource (ATI, document, inspection) porte une liste d'utilisateurs et de droits explicites (`read` , `write` , `comment` , `sign` , `delete`).
2. **RBAC** (Role-Based Access Control) — un rôle = un ensemble figé de permissions, plus une vérification ad hoc sur la propriété de la ressource.

Contraintes : - L'organigramme du Ministère est **stable** : 6 profils suffisent pour 95 % des cas. Ajouter une 7^e catégorie est un événement rare. - La règle métier critique « **un**

opérateur ne voit que ses propres ATI » ne peut pas être exprimée par un simple rôle — elle exige une vérification de propriété. - Le risque IDOR (Insecure Direct Object Reference) sur les opérateurs est classé **CRITIQUE** : un industriel ne doit jamais voir le dossier d'un concurrent.

Décision

Nous adoptons un **RBAC à 6 rôles + un helper de propriété centralisé**.

Les 6 rôles (énum Python `Role`)

```
class Role(StrEnum):
    admin = "admin"
    ministre = "ministre"
    directeur = "directeur"
    instructeur = "instructeur"
    inspecteur = "inspecteur"
    operateur = "operateur"
```

Le helper `check_ati_access`

Centralisé dans `backend/app/routers/ati.py` , importé par tous les routers qui exposent une ressource liée à un ATI (documents, comments, tags, risk, field-history, renew, resubmit, product-qr).

```
def check_ati_access(ati: AgreementTechniqueIndustrielORM, user: User) → None:
    if any(r in PRIVILEGED_ROLES for r in _user_role_values(user)):
        return # admin/ministre/directeur/instructeur/inspecteur : pass-through
    if Role.operateur in _user_role_values(user):
        if ati.created_by != user.username:
            raise HTTPException(403, "Accès refusé à cet ATI.")
        return
    raise HTTPException(403, "Rôle non autorisé.")
```

Règle d'or

Tout endpoint `/pnpi/ati/{id}/*` ouvert à `Role.operateur` doit appeler `check_ati_access` immédiatement après le `db.get` .

Cette règle est documentée dans `CLAUDE.md` du projet et vérifiée à la revue de code (test pattern dans `backend/tests/test_operateur_endpoints.py`).

Conséquences

Positives

- **Lisibilité** : 6 rôles couvrent la matrice fonctionnelle complète (cf. `docs/rbac_matrix.md`). Un nouveau développeur la comprend en une page.
- **Auditabilité** : le contrôle d'accès est concentré dans une fonction. L'audit de sécurité ne cherche pas à vérifier 50 conditions disséminées.
- **Performance** : aucun *join* supplémentaire, aucune lecture de table d'ACL — le contrôle est $O(1)$ en RAM.
- **Compatibilité avec l'organigramme du Ministère** : les rôles épousent exactement les statuts hiérarchiques.

Négatives

- **Granularité limitée** : impossible d'exprimer « cet instructeur précis ne voit que les ATI de la province de l'Estuaire ». La logique province est ajoutée *ad hoc* (champ `User.province`) — c'est une déviation au modèle pur RBAC.
- **Risque humain** : un développeur qui oublie d'appeler `check_ati_access` ouvre un IDOR. Le contrôle reste manuel — pas *enforced* par le framework.
- **Mix enum/string** : la base de données stocke historiquement des rôles en `str` , l'application en `Role` . Le helper `_user_role_values(user)` normalise — mais c'est une dette qu'il faut nettoyer (cf. dette-technique).

Suivi

- **2026-T2** : ajouter un *linter* CI qui détecte les routes `/pnpi/ati/{id}/*` où `Role.operateur` est dans `Depends(require_role)` sans appel à `check_ati_access` dans le corps.
- **2026-T3** : introduire des **scopes territoriaux** (province, département) pour les futurs déploiements multi-tenants CEMAC, sans multiplier les rôles.
- **2026-T4** : étudier l'ajout d'un rôle `auditeur_externe` (Cour des Comptes, IGF) en lecture seule.

Alternatives considérées et rejetées

ACL fines (rejetées)

- **Pour** : flexibilité maximale (« cet inspecteur a délégué ses droits à ce collègue le temps d'un congé »).
- **Contre** : complexité opérationnelle énorme pour une administration publique de taille modeste, surface d'attaque démultipliée, dette de performance (chaque requête nécessite un join sur la table ACL).
- **Verdict** : disproportionné pour le besoin. Les délégations sont gérées séparément (`backend/app/routers/delegations.py`) sans toucher au modèle RBAC.

RBAC à 12 rôles (rejeté)

- **Pour** : couvrir toutes les sous-distinctions (ministre vs cabinet, directeur DGI vs directeur Industrie, etc.).
- **Contre** : explosion combinatoire de la matrice. Aucun gain fonctionnel — les sous-distinctions sont déjà couvertes par les *groupes* (futur), la province et le service.

Comparables

- **France-Connect / Service-Public.fr** : RBAC à 5 rôles + scopes territoriaux. Modèle équivalent.
- **Estonia X-Road** : RBAC + délégations explicites (modèle plus mature que le nôtre — cible 12 mois).
- **Sénégal Numérique** : ACL fines — retours opérationnels mitigés (lourdeur d'administration, courbe d'apprentissage).

Fin de l'ADR-002.